

# SQL

*Fondamenti e Case Study*



Susanna Marras

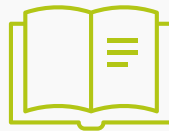
# 1 Cosa si intende per **database**?

Un database è un archivio di dati, raccolti in modo logico su cui è possibile eseguire operazioni di creazione/inserimento, lettura, aggiornamento/modifica ed eliminazione.



CREATE

C



READ

R



UPDATE

U



DELETE

D

Tali dati sono generalmente organizzati secondo delle relazioni e raccolti in tabelle (o entità), colonne (o attributi) e righe.

## 2

## Cos'è un **DBMS**?

Un **DBMS** rappresenta il software con cui è possibile eseguire le operazioni di CRUD sul database.



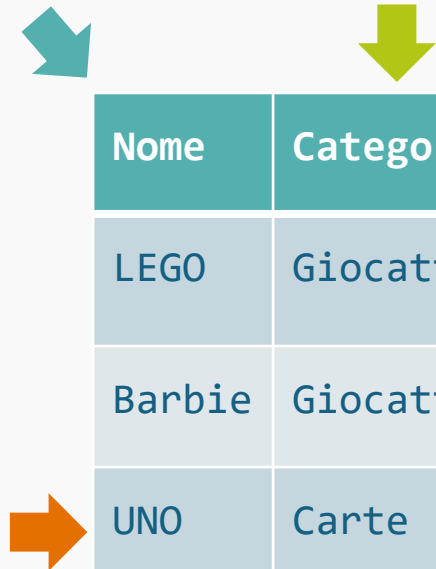
**D**ATABASE **M**ANAGEMENT **S**YSTEM

Quando il database è di tipo relazionale, il corrispondente software è denominato **RDBMS** (Relational Database Management System).

### 3 Qual è l'ordine logico di esecuzione delle clausole in una **SELECT**?

Le principali clausole di uno statement **SELECT** riportate in ordine di esecuzione logica sono:

- **FROM**: indica la tabella da cui selezionare i dati
- **WHERE**: usata nel caso in cui si vogliano filtrare dei risultati, ad esempio selezionando solo dati superiori ad un certo valore.
- **SELECT**: indica le colonne su cui saranno eseguite le funzioni richieste



| Nome   | Categoria  | Qta |
|--------|------------|-----|
| LEGO   | Giocattoli | 10  |
| Barbie | Giocattoli | 8   |
| UNO    | Carte      | 15  |
| Puzzle | Puzzle     | 12  |

## 4 Come funziona lo statement **GROUP BY**?

La funzione GROUP BY consente di raggruppare i dati in base alla condizione indicata.

```
SELECT Categoria,  
SUM(Qta) AS Totale  
FROM Prodotti  
GROUP BY Categoria
```

Prodotti

| Nome   | Categoria  | Qta |
|--------|------------|-----|
| LEGO   | Giocattoli | 10  |
| Barbie | Giocattoli | 8   |
| UNO    | Carte      | 15  |
| Puzzle | Puzzle     | 12  |



| Categoria  | Qta |
|------------|-----|
| Giocattoli | 18  |
| Carte      | 15  |
| Puzzle     | 12  |

5

# Qual è la differenza tra uno schema **OLTP** e uno schema **OLAP**?

## OLTP

*Online Transactional Processing*

Gestione di grandi volumi di transazioni simultanee provenienti da molti utenti, garantendo rapidità e affidabilità.



Si basa tipicamente su database relazionali e operano tramite comandi CRUD, che consentono la modifica dei dati.



Per assicurare la correttezza delle transazioni, segue i principi ACID.



## OLAP

*Online Analytical Processing*

Gestione di operazioni di analisi dei dati, che richiedono generalmente tempi di elaborazione più lunghi.



Si basa su data warehouse, che gestisce volumi di dati provenienti da più sorgenti e li riorganizza per ottimizzare le analisi.



Usa processi ETL, che estraggono i dati dalle sorgenti, li trasformano e li caricano nel data warehouse.



5

## Qual è la differenza tra uno schema **OLTP** e uno schema **OLAP**?



**ATOMICITÀ:** una transazione deve essere completata interamente; se qualcosa fallisce, viene annullata.



**CONSISTENZA:** i dati devono rimanere coerenti prima e dopo la transazione (es. il totale dei saldi coinvolti in un trasferimento rimane invariato).



**ISOLAMENTO:** le transazioni simultanee non devono interferire tra loro, evitando conflitti come la modifica dello stesso record da parte di più utenti.



**DURABILITÀ:** una volta completata, la transazione viene salvata in modo permanente.

## 6 JOIN VS SUBQUERY: a parità di scenario di analisi, qual è la differenza?

|                     | JOIN                                                                                                                                                                  | SUBQUERY                                                                                                                                                                                                      |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cosa fa?            | Unisce una o più tabelle fra loro, sfruttando un attributo comune (PK – primary key).                                                                                 | La subquery è una query annidata dentro un'altra.                                                                                                                                                             |
| Cosa si ottiene?    | Una tabella più grande che consente di visualizzare i dati riportati in tutte le righe delle tabelle interrogate.                                                     | Non è una tabella ma un dato o un insieme di dati specifici da usare ad esempio come filtro nel WHERE.                                                                                                        |
| Quando è più utile? | Quando c'è la necessità di richiamare più attributi da diverse tabelle (ad esempio NomeProdotto, NomeCategoria, NomeBrand dalle Tabelle Prodotto, Categoria e Brand). | Quando si devono selezionare attributi specifici con condizioni particolari (es. solo i prodotti di una determinata categoria venduti in uno specifico anno e il cui fatturato è superiore alla media annua). |



# 7 Cosa si intende per **DML** e **DDL**?

Sono delle istruzioni SQL usate per:

## **DML**

Data **M**anipulation **L**anguage

Gestire i dati, ad esempio attraverso la modifica e l'interrogazione; alcune funzioni sono:

- SELECT
- INSERT
- UPDATE

## **DDL**

Data **D**efinition **L**anguage

Definire la struttura del database, come creazione, modifica ed eliminazione di tabelle, ad es.:

- CREATE
- ALTER
- DROP

## 8 Quali istruzioni possono utilizzare per estrarre l'anno da un campo data?

Per estrarre l'anno si utilizza YEAR(). Di seguito alcuni esempi di utilizzo.

1 `SELECT YEAR('2024-02-06') AS Year;`

2 `SELECT YEAR(OrderDate) AS Year  
FROM SalesOrderHeader;`

# se presente una colonna denominata OrderDate nella tabella in analisi

# 9 Qual è la differenza tra gli operatori logici **AND** e **OR**?

Prodotti

| Nome   | Categoria  | Qta |
|--------|------------|-----|
| LEGO   | Giocattoli | 10  |
| Barbie | Giocattoli | 8   |
| UNO    | Carte      | 15  |
| Puzzle | Puzzle     | 12  |

**AND**: Si usa per filtrare secondo due o più condizioni. Restituisce VERO se TUTTE le condizioni sono VERE.

```
SELECT Nome
FROM Prodotti
WHERE Categoria = 'Giocattoli' AND Qta = 8
```

| Nome   |
|--------|
| Barbie |

**OR**: Si usa per filtrare secondo due o più condizioni. Restituisce VERO se ALMENO UNA condizione è VERA.

```
SELECT Nome
FROM Prodotti
WHERE Categoria = 'Carte' OR
      Categoria = 'VideoGames'
```

| Nome |
|------|
| UNO  |

# 10 È possibile **innestare** una query nella clausola **SELECT**?

Sì, è possibile nel caso della **query innestata scalare indipendente**, in cui la query innestata (inner query) può essere usata indipendentemente e restituisce un singolo valore che viene utilizzato come input dalla query esterna (outer query).

```
SELECT Nome, Qta,  
(SELECT AVG(Qta) FROM  
Prodotti) AS MediaQta  
FROM Prodotti  
WHERE Qta > (SELECT  
AVG(Qta) FROM Prodotti)
```

## Prodotti

| Nome   | Categoria  | Qta |
|--------|------------|-----|
| LEGO   | Giocattoli | 10  |
| Barbie | Giocattoli | 8   |
| UNO    | Carte      | 15  |
| Puzzle | Puzzle     | 12  |



| Nome   | Qta | MediaQta |
|--------|-----|----------|
| UNO    | 15  | 11.25    |
| Puzzle | 12  | 11.25    |

# 11 Qual è la differenza tra l'operatore logico **OR** e l'operatore logico **IN**?

Gli operatori **OR** e **IN** sono equivalenti e si utilizzano per cercare la corrispondenza di un valore ad una lista.

Entrambi restituiscono VERO se l'argomento è uguale alle espressioni elencate.

- **OR** richiede di specificare ogni volta tutti i valori con cui cercare corrispondenza:

```
SELECT Nome
FROM Prodotti
WHERE Categoria = 'Carte' OR Categoria = 'Puzzle'
```

- **IN** risulta essere più efficace nel caso in cui la corrispondenza viene eseguita con un numero maggiore di valori

```
SELECT Qta
FROM Prodotti
WHERE Categoria IN ('Carte' , 'Puzzle')
```

| Prodotti |            |     |
|----------|------------|-----|
| Nome     | Categoria  | Qta |
| LEGO     | Giocattoli | 10  |
| Barbie   | Giocattoli | 8   |
| UNO      | Carte      | 15  |
| Puzzle   | Puzzle     | 12  |



| Qta |
|-----|
| 15  |
| 12  |

# 12 L'operatore logico **BETWEEN** include anche gli estremi del range specificato?

Sì, a contrario del solo **AND**, che include gli estremi solo se si usano le forme  $\geq$  e  $\leq$ .

Prodotti

| Nome   | Categoria  | Qta |
|--------|------------|-----|
| LEGO   | Giocattoli | 10  |
| Barbie | Giocattoli | 8   |
| UNO    | Carte      | 15  |
| Puzzle | Puzzle     | 12  |

**BETWEEN**: il range è riportato a sinistra e a destra dell'operatore logico AND

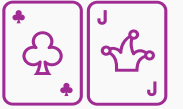
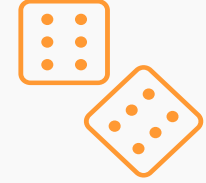
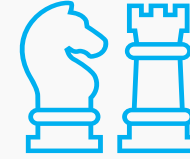
```
SELECT Nome
FROM Prodotti
WHERE Qta BETWEEN 1 AND 10
```

| Qta |
|-----|
| 10  |
| 8   |

**AND**: se non specificato, non include gli estremi

```
SELECT Qta
FROM Prodotti
WHERE Qta>1 AND Qta<10
```

| Qta |
|-----|
| 8   |



# CASE STUDY

“ToysGroup è un’azienda che distribuisce articoli (**giocattoli**) in diverse aree geografiche del mondo.

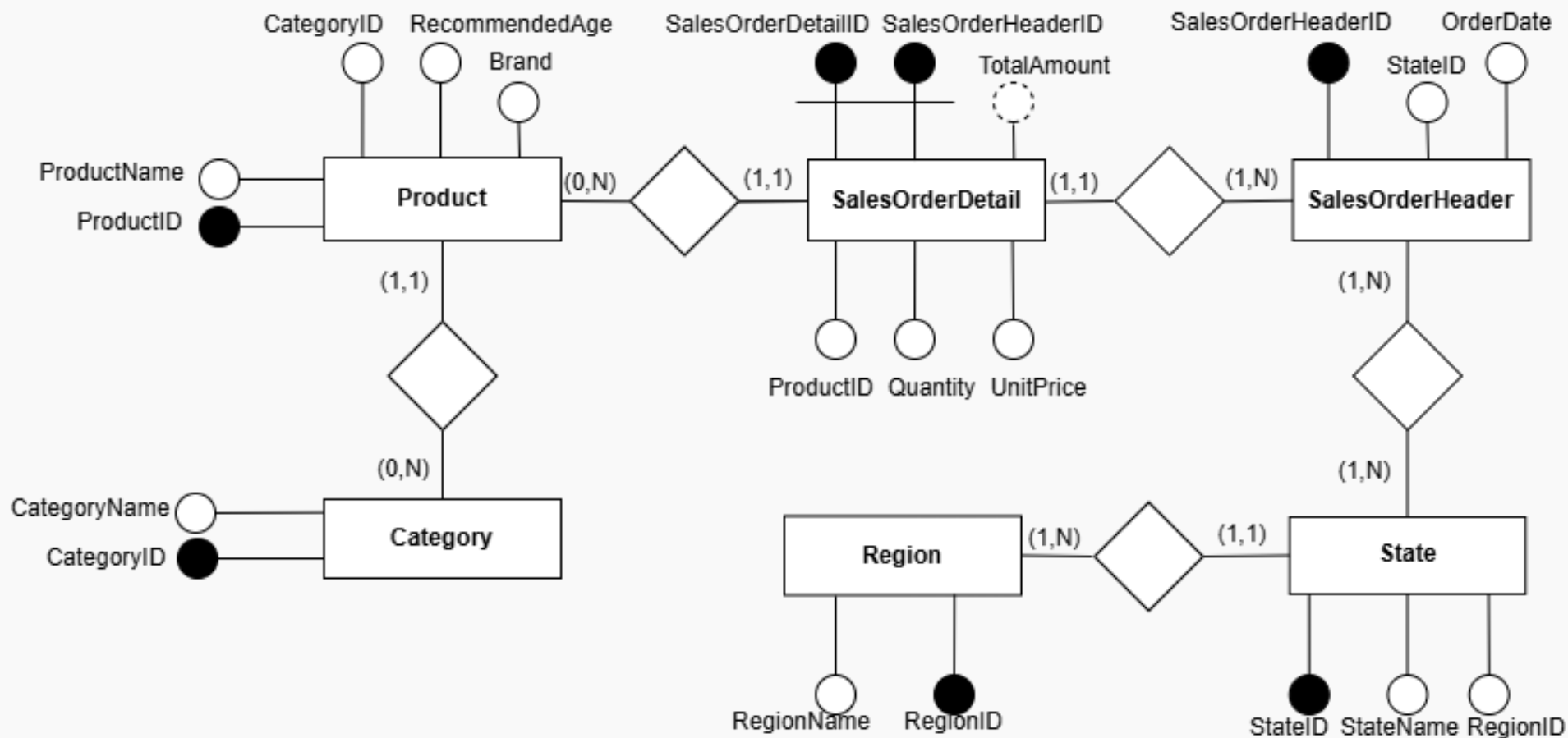
I **prodotti** sono classificati in **categorie** e i mercati di riferimento dell’azienda sono classificati in **regioni di vendita.**”

# Attività svolte:

- 1 Creazione Modello Concettuale e Logico, con identificazione di Entità, Attributi e Primary Keys (anche note come PK o chiavi primarie)
- 2 Creazione Database e tabelle
- 3 Inserimento dati nelle tabelle create
- 4 Analisi dei dati



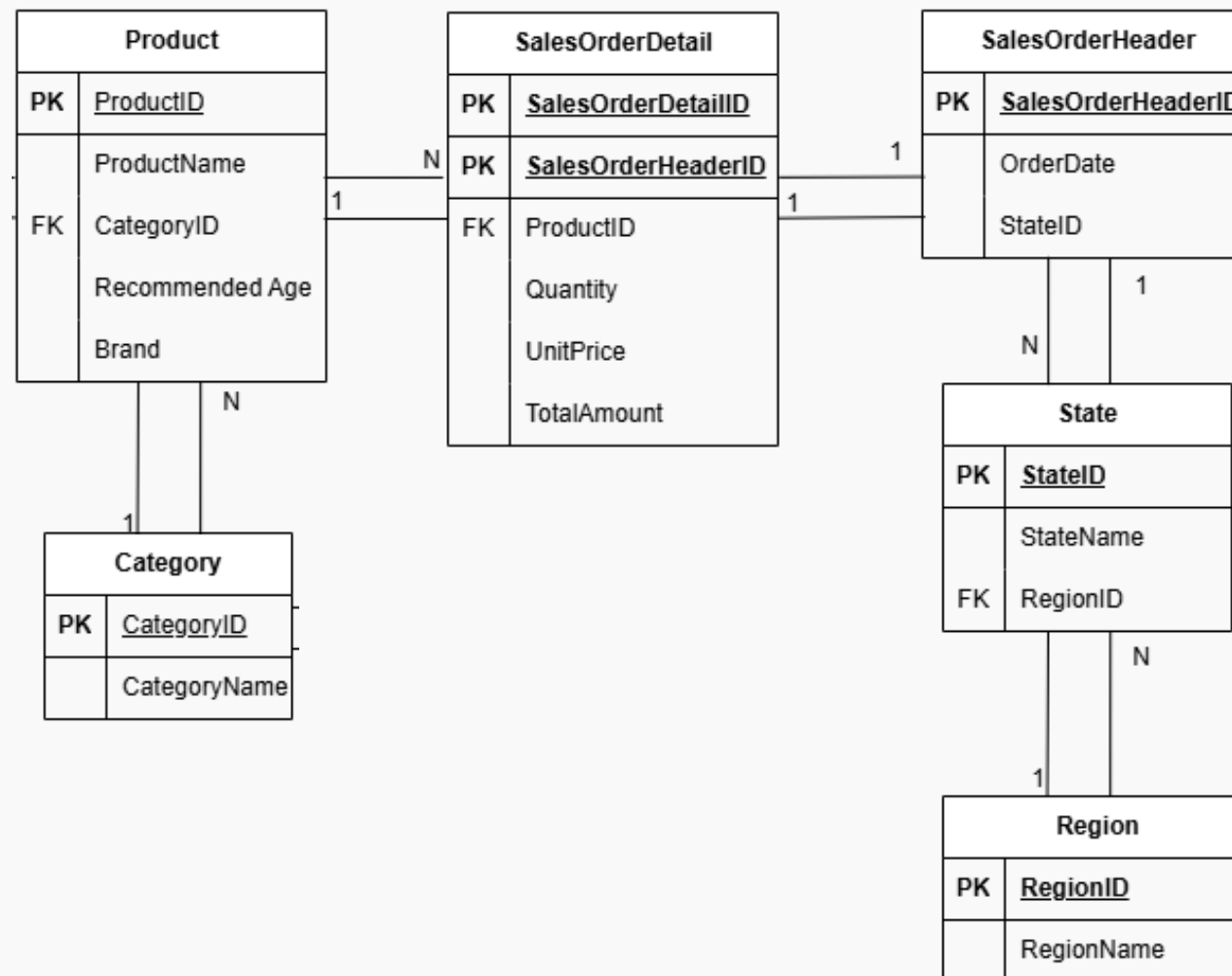
# 1.1 MODELLO CONCETTUALE



# 1.2 MODELLO LOGICO



| Entità           | Attributi                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------|
| Category         | CategoryID<br>CategoryName                                                                    |
| Product          | ProductID<br>ProductName<br>CategoryID<br>RecommendedAge<br>Brand                             |
| SalesOrderDetail | SalesOrderDetailID<br>SalesOrderHeaderID<br>ProductID<br>Quantity<br>UnitPrice<br>TotalAmount |
| SalesOrderHeader | SalesOrderHeaderID<br>OrderDate<br>StateID                                                    |
| State            | StateID<br>StateName<br>RegionID                                                              |
| Region           | RegionID<br>RegionName                                                                        |



## 2

# CREAZIONE DEL DATABASE E DELLE TABELLE

Creazione del **Database** (denominato ToysGroup) e successiva creazione delle **Tabelle**:

- Category
- Product
- SalesOrderDetail
- SalesOrderHeader
- State
- Region

Per ciascuna tabella sono state identificate le primary keys e i formati di ciascun attributo

```
# Creazione database
CREATE DATABASE ToysGroup;
```

```
# Selezione del database
USE ToysGroup;
```

```
# Creazione tabelle
CREATE TABLE
Category (CategoryID INT AUTO_INCREMENT PRIMARY KEY,
CategoryName VARCHAR(100) );
```

```
CREATE TABLE Product
(ProductID INT AUTO_INCREMENT PRIMARY KEY,
ProductName VARCHAR(100),
CategoryID INT,
RecommendedAge INT, Brand VARCHAR(100),
CONSTRAINT FK_Product_CategoryID FOREIGN KEY(CategoryID)
REFERENCES Category(CategoryID));
```

...

# 3 INSERIMENTO DATI NELLE TABELLE

Popolamento di ciascuna **tabella** con dati congruenti a quanto specificato nella creazione delle stesse

```
INSERT INTO Category (CategoryName) VALUES  
( 'Building Sets' ),  
( 'Dolls & Plush' ),  
( 'Board Games' );
```

```
INSERT INTO Product  
(ProductName, CategoryID, RecommendedAge, Brand) VALUES  
( 'Galactic Starship LEGO', 1, 9, 'LEGO' ),  
( 'Talking Teddy Bear', 2, 3, 'HuggyTime' ),  
( 'Mystery Mansion Clue', 3, 10, 'Hasbro' ),  
( 'Space Explorer LEGO', 1, 9, 'LEGO' ),  
( 'Talking Teddy Bear', 2, 3, 'HuggyTime' ),  
( 'Junior Chess Set', 3, 7, 'SmartKids' ),  
( 'RC Racing Car', 1, 8, 'SpeedMasters' );  
...
```

# 4 ANALISI DATI



## 4 ANALISI DATI

**Q.1** Verificare che i campi definiti come PK siano univoci. In altre parole, scrivi una query per determinare l'univocità dei valori di ciascuna PK (una query per tabella implementata).

Per tale verifica, si raggruppano i valori della PK si conta il numero di volte in cui si ripetono.

- Se la query non restituisce alcuna riga, significa che nessun valore compare più di una volta e quindi la PK è correttamente univoca.
- Se la query restituisce una o più righe, allora esistono valori duplicati e quindi si perde l'univocità caratteristica della PK.

```
SELECT CategoryID,  
COUNT(CategoryID) AS CountCategoryID  
FROM Category  
GROUP BY CategoryID  
HAVING COUNT(CategoryID)>1;
```

| CategoryID | CategoryName  |
|------------|---------------|
| 1          | Building Sets |
| 2          | Dolls & Plush |
| 3          | Board Games   |



| CategoryID | CountCategoryID |
|------------|-----------------|
|            |                 |

# 4

## ANALISI DATI

Q.2

Esporre l'elenco delle transazioni indicando nel result set:

- il codice documento,
- la data,
- il nome del prodotto,
- la categoria del prodotto,
- il nome dello stato,
- il nome della regione di vendita e
- un campo booleano valorizzato in base alla condizione che siano passati più di 180 giorni dalla data vendita o meno (>180 -> True, <= 180 -> False)

## 4 ANALISI DATI

Q.2

```
SELECT SalesOrderHeader.SalesOrderHeaderID AS OrderHeaderID,  
OrderDate,Product.ProductName AS ProductName,  
Category.CategoryName AS CategoryName,State.StateName AS State,  
Region.RegionName AS Region,DATEDIFF(CURDATE(),OrderDate) >180 AS OlderThan180Days  
FROM SalesOrderHeader  
INNER JOIN SalesOrderDetail ON SalesOrderHeader.SalesOrderHeaderID =  
SalesOrderDetail.SalesOrderHeaderID  
INNER JOIN Product ON SalesOrderDetail.ProductID = Product.ProductID  
INNER JOIN Category ON Product.CategoryID = Category.CategoryID  
INNER JOIN State ON SalesOrderHeader.StateID = State.StateID  
INNER JOIN Region ON State.RegionID =  
Region.RegionID
```



| OrderHeaderID | OrderDate  | ProductName            | CategoryName  | State  | Region       | OlderThan180Days |
|---------------|------------|------------------------|---------------|--------|--------------|------------------|
| 3             | 15/06/2025 | Mystery Mansion Clue   | Board Games   | France | WestEurope   | 1                |
| 3             | 15/06/2025 | Galactic Starship LEGO | Building Sets | France | WestEurope   | 1                |
| 1             | 26/02/2026 | Mystery Mansion Clue   | Board Games   | USA    | NorthAmerica | 0                |
| ...           | ...        | ...                    | ...           | ...    | ...          | ...              |



## 4 ANALISI DATI

Q.3

Esporre l'elenco dei prodotti che hanno venduto, in totale, una quantità maggiore della media delle vendite realizzate nell'ultimo anno censito. (ogni valore della condizione deve risultare da una query e non deve essere inserito a mano). Nel result set devono comparire solo il codice prodotto e il totale venduto.

```
SELECT ProductID, SUM(Quantity) AS TotalQuantitySold
FROM SalesOrderDetail
INNER JOIN SalesOrderHeader ON SalesOrderHeader.SalesOrderHeaderID =
    SalesOrderDetail.SalesOrderHeaderID
WHERE YEAR(SalesOrderHeader.OrderDate) = (SELECT MAX(YEAR(OrderDate))
FROM SalesOrderHeader)
GROUP BY ProductID HAVING SUM(Quantity) > (
    SELECT AVG(Quantity) FROM SalesOrderDetail
    INNER JOIN SalesOrderHeader ON
        SalesOrderHeader.SalesOrderHeaderID =
            SalesOrderDetail.SalesOrderHeaderID
    WHERE YEAR(SalesOrderHeader.OrderDate) = (
        SELECT MAX(YEAR(OrderDate)) FROM
            SalesOrderHeader))
ORDER BY TotalQuantitySold DESC
```



| ProductID | TotalQuantitySold |
|-----------|-------------------|
| 1         | 6                 |

## 4 ANALISI DATI

Q.4 Esporre l'elenco dei soli prodotti venduti e per ognuno di questi il fatturato totale per anno.

```
SELECT SalesOrderDetail.ProductID,  
       YEAR(OrderDate),  
       SUM(TotalAmount) AS TotalRevenue  
FROM SalesOrderDetail  
INNER JOIN SalesOrderHeader ON  
    SalesOrderHeader.SalesOrderHeaderID =  
    SalesOrderDetail.SalesOrderHeaderID  
GROUP BY ProductID, YEAR(OrderDate)
```



| ProductID | Year | TotalRevenue |
|-----------|------|--------------|
| 1         | 2026 | 339.99       |
| 2         | 2023 | 60.00        |
| 3         | 2026 | 50.00        |

## 4

## ANALISI DATI

Q.5

Esporre il fatturato totale per stato per anno. Ordina il risultato per data e per fatturato decrescente.

```
SELECT State.StateName AS State,  
       YEAR(OrderDate) AS OrderYear,  
       SUM(SalesOrderDetail.TotalAmount) AS  
         TotalRevenue  
FROM SalesOrderDetail  
INNER JOIN SalesOrderHeader ON  
    SalesOrderHeader.SalesOrderHeaderID =  
    SalesOrderDetail.SalesOrderHeaderID  
LEFT JOIN State ON SalesOrderHeader.StateID  
    = State.StateID  
GROUP BY State.StateName, OrderDate  
ORDER BY YEAR(OrderDate) DESC,  
         TotalRevenue DESC
```



| ProductID | Year | TotalRevenue |
|-----------|------|--------------|
| USA       | 2026 | 389.99       |
| USA       | 2025 | 675.00       |
| France    | 2025 | 540.00       |
| Italy     | 2023 | 60.00        |

## 4 ANALISI DATI

Q.6 Rispondere alla seguente domanda:  
qual è la categoria di articoli maggiormente richiesta dal mercato?

```
SELECT Category.CategoryID,  
       SUM(SalesOrderDetail.Quantity) AS  
       MostRequestedCategory  
FROM SalesOrderDetail  
INNER JOIN Product ON SalesOrderDetail.ProductID =  
       Product.ProductID  
LEFT JOIN Category ON Product.CategoryID =  
       Category.CategoryID  
GROUP BY CategoryID  
ORDER BY MostRequestedCategory DESC
```



| CategoryID | MostRequestedCategory |
|------------|-----------------------|
| 1          | 31                    |
| 3          | 4                     |
| 2          | 2                     |

## 4 ANALISI DATI

Q.7 Rispondere alla seguente domanda: quali sono i prodotti invenduti?  
Proponi due approcci risolutivi differenti.

1

```
SELECT Product.ProductID
FROM Product
LEFT JOIN SalesOrderDetail ON SalesOrderDetail.ProductID =
                             Product.ProductID
WHERE SalesOrderDetail.ProductID IS NULL
ORDER BY Product.ProductID
```

2

```
SELECT ProductID
FROM Product
WHERE ProductID NOT IN (SELECT DISTINCT ProductID
                        FROM SalesOrderDetail)
ORDER BY ProductID;
```



| ProductID |
|-----------|
| 5         |
| 6         |
| 7         |

## 4 ANALISI DATI

**Q.8** Creare una vista sui prodotti in modo tale da esporre una “versione denormalizzata” delle informazioni utili (codice prodotto, nome prodotto, nome categoria)

```
CREATE VIEW ListProductCategory AS (  
  
SELECT Product.ProductID,  
Product.ProductName,  
Category.CategoryName  
FROM Product  
LEFT JOIN Category ON  
    Product.CategoryID =  
    Category.CategoryID);  
  
SELECT * FROM  
    toysgroup.listproductcategory;
```



| ProductID | ProductName            | CategoryName  |
|-----------|------------------------|---------------|
| 1         | Galactic Starship LEGO | Building Sets |
| 2         | Talking Teddy Bear     | Dolls & Plush |
| 3         | Mystery Mansion Clue   | Board Games   |
| ...       | ...                    | ...           |

## 4

## ANALISI DATI

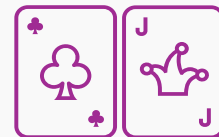
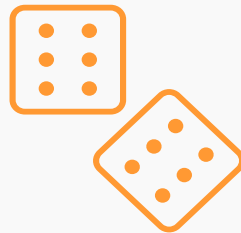
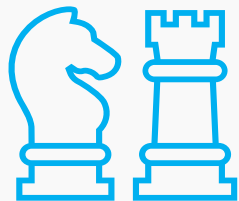
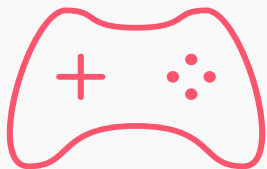
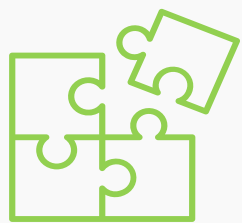
Q.9

Creare una vista per le informazioni geografiche

```
CREATE VIEW Geography AS (  
  
SELECT State.StateID AS StateID,  
       Region.RegionID AS RegionID,  
       StateName AS State,  
       Region.RegionName AS Region  
FROM State  
INNER JOIN Region ON State.RegionID =  
                     Region.RegionID);  
  
SELECT * FROM toysgroup.geography;
```



| StateID | RegionID | State  | Region       |
|---------|----------|--------|--------------|
| 1       | 1        | France | WestEurope   |
| 2       | 2        | USA    | NorthAmerica |
| 3       | 3        | Italy  | SouthEurope  |



---

*Grazie*